



Recitation 9

Recitation overview

- Program arguments
- Dynamic memory allocation

Recitation overview

- Program arguments
- Dynamic memory allocation

Command line arguments

Until now we referred to the prototype of `main` as follows:

```
int main () ;
```

but its complete prototype is:

```
int main (int argc, char *argv[]) ;
```

or

```
int main (int argc, char **argv) ;
```

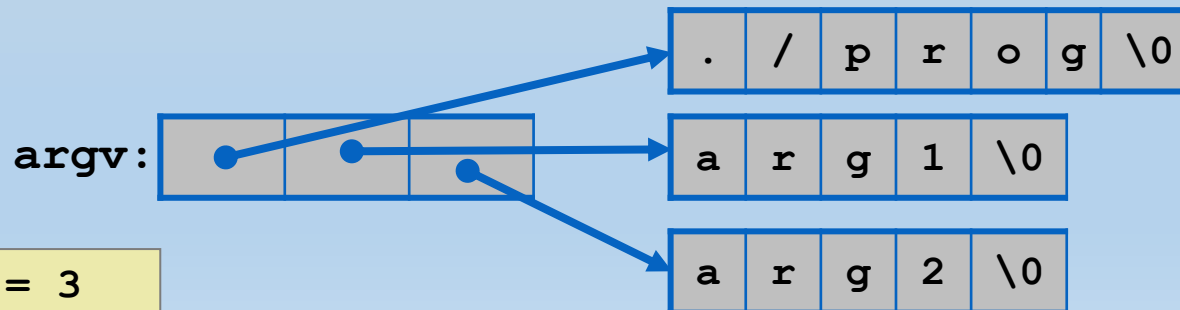
- `argc` stands for *argument count* – the number of arguments
- `argv` stands for *argument vector* – an array of strings

Command line arguments

`main()` argument **`argv`** is of type **`char**`** – an array of pointers, pointing to the starting points of strings (allocated in stack frame of **`main`**)

- **`argv[0]`** (type **`char*`**) is the string holding the program's name
- **`argv[i]`** (type **`char*`**) ($i > 0$) is the string holding the i^{th} argument
- **`argv[i][j]`** (type **`char`**) ($i > 0, j \geq 0$) is the $(j-1)^{\text{th}}$ character of **`argv[i]`**

```
==> ./prog arg1 arg2
```



```
argc = 3
```

Recitation overview

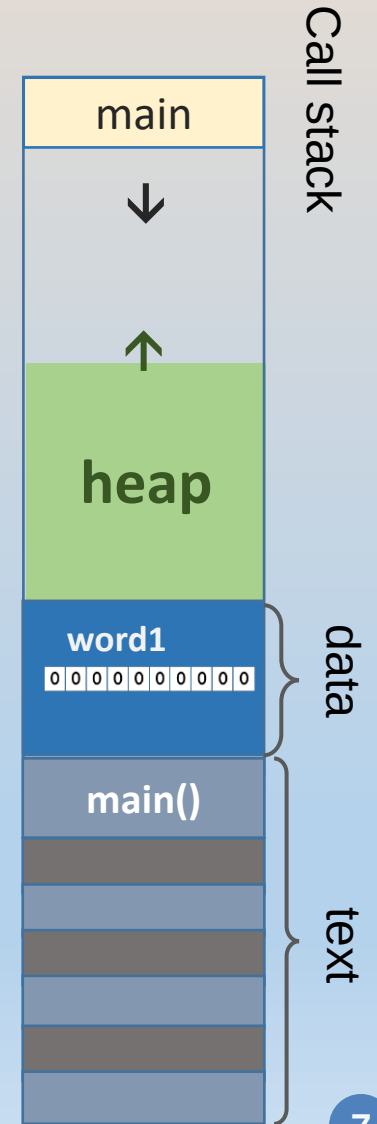
- Program arguments
- Dynamic memory allocation

Dynamic memory allocation

How does it work?

- Allocate continuous block of memory using a memory allocation library function (`malloc`, `calloc`, `realloc`)
- Memory block can be used globally in the program, because it does not get automatically freed
- Free block of memory by calling a memory freeing library function (such as `free()`)

➔ Relevant system library: `stdlib.h`



malloc

```
void* malloc(size_t)
```

- Input: number of **bytes** to allocate
(`size_t` is an integral type returned by `sizeof`)
- Output:
 - If successful – start address (`void*`) to the memory block allocated
 - If failed – returns NULL **← always check!!**
(recall: NULL is 0 in pointer type)

free

`void free(void*)`

- Input: the address of a previously-allocated block of memory
- Freeing memory **does not flag the contents as “freed”**; it just changes the program’s memory allocation tables to indicate this block can be allocated later
- **Common mistakes** (resulting in runtime error):
 - freeing the same block more than once
 - using free on a pointer to an address that has not been allocated
 - **using a pointer after it has been freed**
(will not actually trigger an error message !!)